Vector Field - Reactive Algorithms Pt 5

A Stream with a Spring

This one is going to have less visualizations than the first.

Max Flow Finder

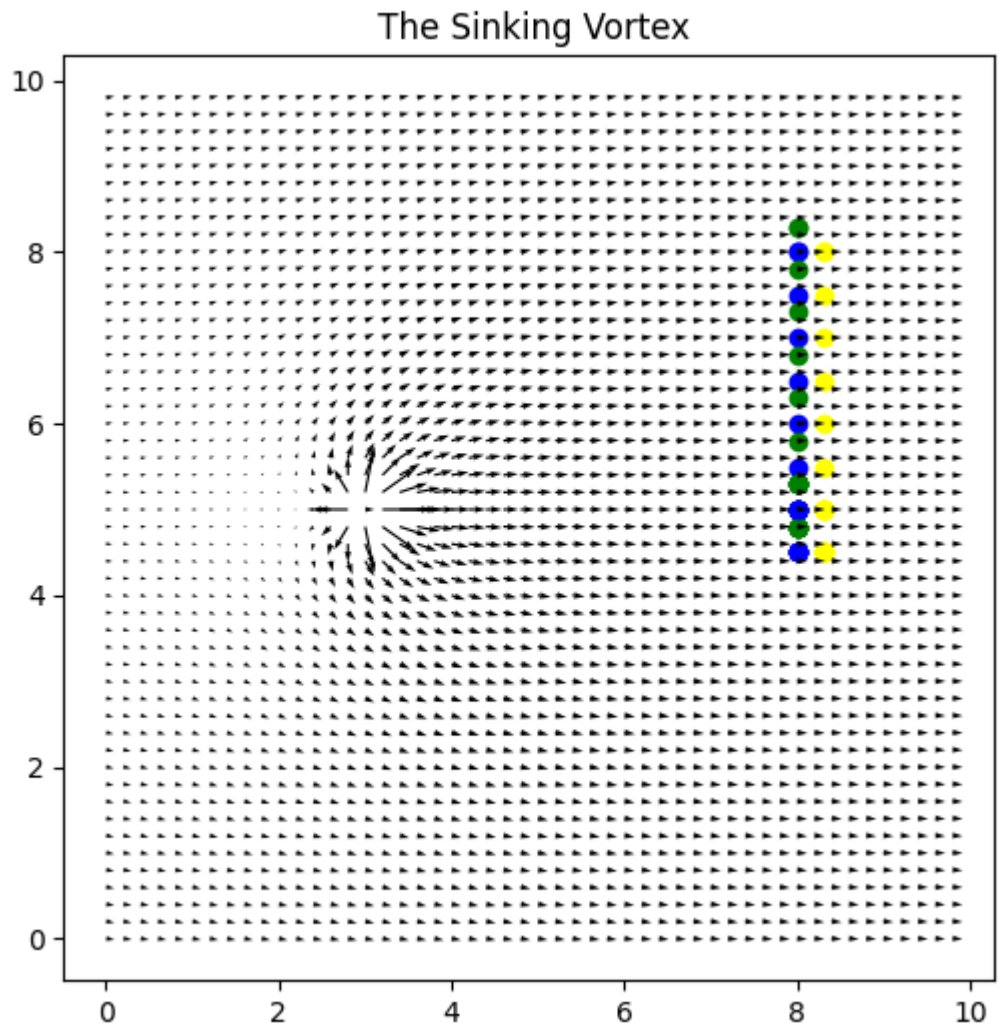
In [1]:

```python
## 1. Import Libraries
import numpy as np
import matplotlib.pyplot as plt

# 2. Define constants
x_start = y_start = 0
x_end = y_end = 10
quiver_step_size = 0.2

x_source = 3
y_source = 5

robot_1_start = [8,8]
robot_2_offset = [0.3,0]
robot_3_offset = [0,0.3]

robot_step_size = 0.5



def spring_stream(x, y, x_source, y_source):

    # Establishing a Uniform Flow
    u = np.ones_like(x)
    v = np.zeros_like(x)


    # Calculating A Source
    x_centre = x - x_source
    y_centre = y - y_source
    r = np.sqrt(x_centre**2 + y_centre**2)
    r[r==0] = 1e-6

    # Adding the uniform flow to the spring source
    u = u + x_centre / r**2
    v = v + y_centre / r**2

    return u, v

def robot_readings(x, y, x_source, y_source):

    # Centring over sink
    x_centre = x - x_source
```

```
    y_centre = y - y_source

    # Calculate the distance r from the center
    dist = np.sqrt((x-x_source)**2 + (y-y_source)**2)

    # Establishing a Uniform Flow
    u = np.ones_like(x)
    v = np.zeros_like(x)

    # Adding the uniform flow to the spring source
    u = u + x_centre / dist**2
    v = v + y_centre / dist**2

    return u, v

def composite_vector(u,v):

    mag = np.sqrt(u**2 + v**2)
    angle = np.arctan2(v2,u2)

    return mag, angle

def plot_robot_position(pos1,pos2,pos3):

    plt.scatter(pos1[0],pos1[1], marker='o', color='blue')
    plt.scatter(pos2[0],pos2[1], marker='o', color = 'yellow')
    plt.scatter(pos3[0],pos3[1], marker='o', color = 'green')


def update_robot_position(pos1,pos2,pos3, slope_y, step_size):

    if np.sign(slope_y)>0:
        pos1[1] += step_size
        pos2[1] += step_size
        pos3[1] += step_size
    else:
        pos1[1] -= step_size
        pos2[1] -= step_size
        pos3[1] -= step_size

    return pos1, pos2, pos3

# 4. Initialize
```

```
87
88  # Initialize The Sinking Vortex Field
89  x = np.arange(x_start, x_end, quiver_step_size)
90  y = np.arange(y_start, y_end, quiver_step_size)
91  X, Y = np.meshgrid(x, y)
92  u, v = spring_stream(X, Y, x_source, y_source)
93
94  # Initialize the Robot's positions
95  pos1=robot_1_start
96  pos2=[(robot_1_start[0] + robot_2_offset[0]),(robot_1_start[1] + robot_2_offset[1])]
97  pos3=[(robot_1_start[0] + robot_3_offset[0]),(robot_1_start[1] + robot_3_offset[1])]
98
99  # Initialize output Plot
100 plt.figure(figsize=(6, 6))
101 plot_robot_position(pos1,pos2,pos3)
102
103 # 5. Main
104
105 for _ in range(1,20):
106     # Check Break Criteria
107
108     # Find u,v, angle and magnitude at each Robot
109
110     [u1, v1] = robot_readings(pos1[0],pos1[1], x_source,y_source)
111     [u2, v2] = robot_readings(pos2[0],pos2[1], x_source,y_source)
112     [u3, v3] = robot_readings(pos3[0],pos3[1], x_source,y_source)
113
114     [mag1, angle1] = composite_vector(u1,v1)
115     [mag2, angle2] = composite_vector(u2,v2)
116     [mag3, angle3] = composite_vector(u3,v3)
117
118     # Plot Robot Position
119     plot_robot_position(pos1,pos2,pos3)
120
121     # Update Robot Position
122     #angle = -(angle1 + angle2 + angle3)/3
123     #slope_x = (mag2 - mag1) / robot_2_offset[0]
124     slope_y = (mag3 - mag1) / robot_3_offset[1]
125     #angle = np.arctan2(slope_y,slope_x) #+ np.pi/2
126
127     # Update Robot Position
128
129     [pos1, pos2, pos3] = update_robot_position(pos1,pos2,pos3,slope_y, robot_step_size)
```

```
130
131
132 # 6. Plot Outcome
133
134
135 plt.quiver(x, y, u, v)
136 plt.title(' The Sinking Vortex')
137 #plot_extra_features(x, y, u, v, x_source, y_source)
138 plt.show()
139
```



Source Finder

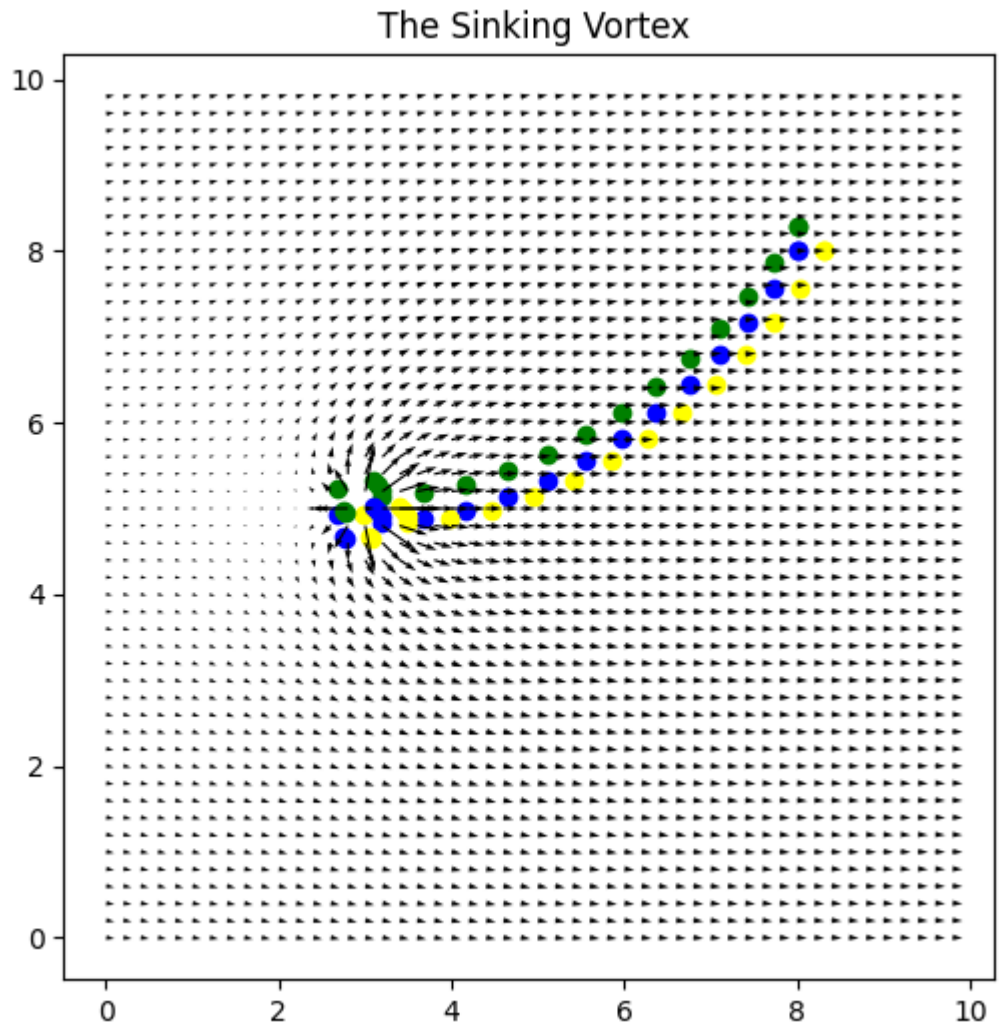
In [4]:

```python
## 1. Import Libraries
import numpy as np
import matplotlib.pyplot as plt

# 2. Define constants
x_start = y_start = 0
x_end = y_end = 10
quiver_step_size = 0.2

x_source = 3
y_source = 5

robot_1_start = [8,8]
robot_2_offset = [0.3,0]
robot_3_offset = [0,0.3]

robot_step_size = 0.5



def spring_stream(x, y, x_source, y_source):

    # Establishing a Uniform Flow
    u = np.ones_like(x)
    v = np.zeros_like(x)


    # Calculating A Source
    x_centre = x - x_source
    y_centre = y - y_source
    r = np.sqrt(x_centre**2 + y_centre**2)
    r[r==0] = 1e-6

    # Adding the uniform flow to the spring source
    u = u + x_centre / r**2
    v = v + y_centre / r**2

    return u, v

def robot_readings(x, y, x_source, y_source):

    # Centring over sink
    x_centre = x - x_source
```

```
    y_centre = y - y_source

    # Calculate the distance r from the center
    dist = np.sqrt((x-x_source)**2 + (y-y_source)**2)

    # Establishing a Uniform Flow
    u = np.ones_like(x)
    v = np.zeros_like(x)

    # Adding the uniform flow to the spring source
    u = u + x_centre / dist**2
    v = v + y_centre / dist**2

    return u, v

def composite_vector(u,v):

    mag = np.sqrt(u**2 + v**2)
    angle = np.arctan2(v2,u2)

    return mag, angle

def plot_robot_position(pos1,pos2,pos3):

    plt.scatter(pos1[0],pos1[1], marker='o', color='blue')
    plt.scatter(pos2[0],pos2[1], marker='o', color = 'yellow')
    plt.scatter(pos3[0],pos3[1], marker='o', color = 'green')


def update_robot_position(pos1,pos2,pos3, angle, step_size):

    pos1[0] += step_size*np.cos(angle)
    pos1[1] += step_size*np.sin(angle)
    pos2[0] += step_size*np.cos(angle)
    pos2[1] += step_size*np.sin(angle)
    pos3[0] += step_size*np.cos(angle)
    pos3[1] += step_size*np.sin(angle)

    return pos1, pos2, pos3

# 4. Initialize

# Initialize The Sinking Vortex Field
```

```
x = np.arange(x_start, x_end, quiver_step_size)
y = np.arange(y_start, y_end, quiver_step_size)
X, Y = np.meshgrid(x, y)
u, v = spring_stream(X, Y, x_source, y_source)

# Initialize the Robot's positions
pos1=robot_1_start
pos2=[(robot_1_start[0] + robot_2_offset[0]),(robot_1_start[1] + robot_2_offset[1])]
pos3=[(robot_1_start[0] + robot_3_offset[0]),(robot_1_start[1] + robot_3_offset[1])]

# Initialize output Plot
plt.figure(figsize=(6, 6))
plot_robot_position(pos1,pos2,pos3)

# 5. Main

for _ in range(1,20):
    # Check Break Criteria

    # Find u,v, angle and magnitude at each Robot

    [u1, v1] = robot_readings(pos1[0],pos1[1], x_source,y_source)
    [u2, v2] = robot_readings(pos2[0],pos2[1], x_source,y_source)
    [u3, v3] = robot_readings(pos3[0],pos3[1], x_source,y_source)

    [mag1, angle1] = composite_vector(u1,v1)
    [mag2, angle2] = composite_vector(u2,v2)
    [mag3, angle3] = composite_vector(u3,v3)

    # Plot Robot Position
    plot_robot_position(pos1,pos2,pos3)

    # Update Robot Position
    #angle = -(angle1 + angle2 + angle3)/3
    slope_x = (mag2 - mag1) / robot_2_offset[0]
    slope_y = (mag3 - mag1) / robot_3_offset[1]
    angle = np.arctan2(slope_y,slope_x) #+ np.pi/2

    # Update Robot Position

    [pos1, pos2, pos3] = update_robot_position(pos1,pos2,pos3,angle, robot_step_size)
```

```
# 6. Plot Outcome


plt.quiver(x, y, u, v)
plt.title(' The Sinking Vortex')
#plot_extra_features(x, y, u, v, x_source, y_source)
plt.show()

```



Fastest Flow Finder - 2 steps, straight line

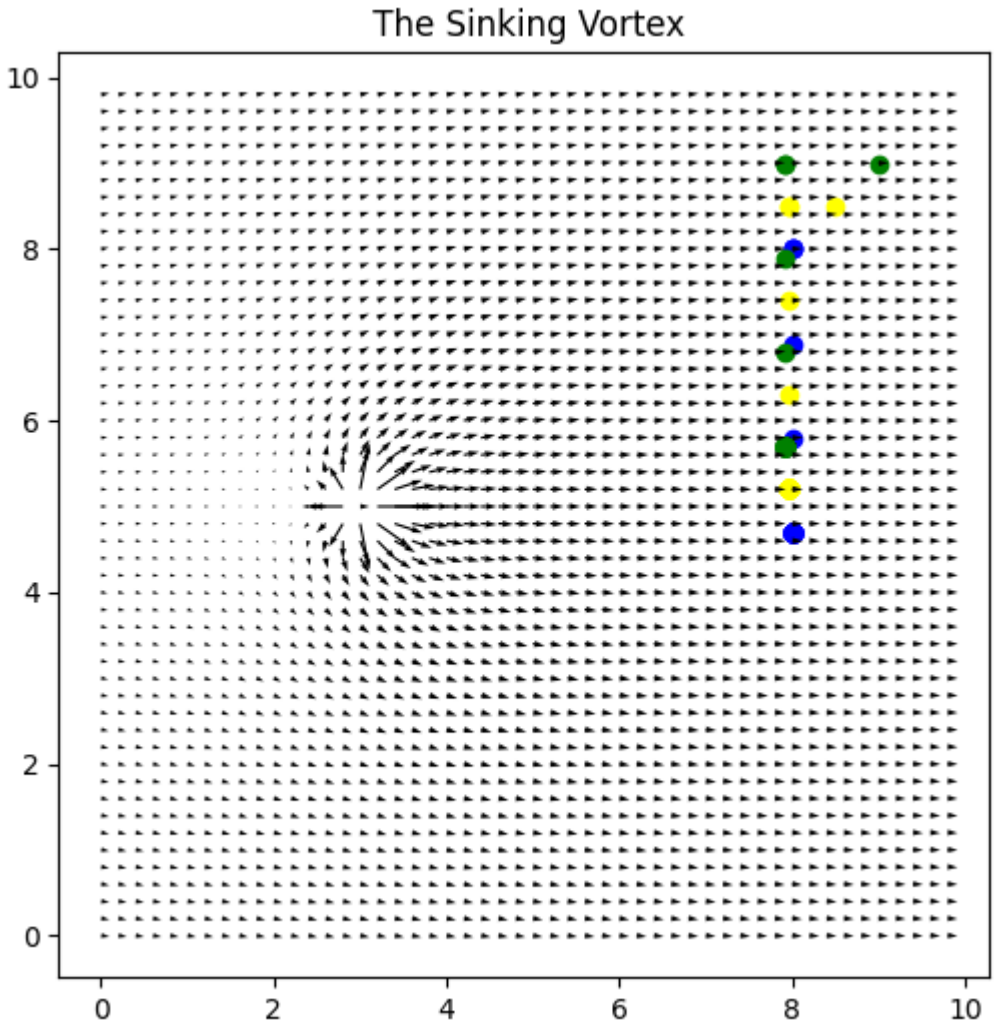
In [2]:

```python
## 1. Import Libraries
import numpy as np
import matplotlib.pyplot as plt

# 2. Define constants
x_start = y_start = 0
x_end = y_end = 10
quiver_step_size = 0.2

x_source = 3
y_source = 5

robot_1_start = [8,8]
robot_2_offset = [0.5,0.5]
robot_3_offset = [1,1]
robot_offset = 0.5

robot_step_size = 1.1



def spring_stream(x, y, x_source, y_source):

    # Establishing a Uniform Flow
    u = np.ones_like(x)
    v = np.zeros_like(x)


    # Calculating A Source
    x_centre = x - x_source
    y_centre = y - y_source
    r = np.sqrt(x_centre**2 + y_centre**2)
    r[r==0] = 1e-6

    # Adding the uniform flow to the spring source
    u = u + x_centre / r**2
    v = v + y_centre / r**2

    return u, v

def robot_readings(x, y, x_source, y_source):

    # Centring over sink
```

```
    x_centre = x - x_source
    y_centre = y - y_source

    # Calculate the distance r from the center
    dist = np.sqrt((x-x_source)**2 + (y-y_source)**2)

    # Establishing a Uniform Flow
    u = np.ones_like(x)
    v = np.zeros_like(x)

    # Adding the uniform flow to the spring source
    u = u + x_centre / dist**2
    v = v + y_centre / dist**2

    return u, v

def composite_vector(u,v):

    mag = np.sqrt(u**2 + v**2)
    angle = np.arctan2(v2,u2)

    return mag, angle

def plot_robot_position(pos1,pos2,pos3):

    plt.scatter(pos1[0],pos1[1], marker='o', color='blue')
    plt.scatter(pos2[0],pos2[1], marker='o', color = 'yellow')
    plt.scatter(pos3[0],pos3[1], marker='o', color = 'green')


def update_robot_position(pos1,pos2,pos3, slope_y1, slope_y2, step_size):

    if np.sign(slope_y1)>0 and np.sign(slope_y2)>0 :
        pos1[1] += step_size
        pos2[1] += step_size
        pos3[1] += step_size
    elif np.sign(slope_y1)<0 and np.sign(slope_y2)<0 :
        pos1[1] -= step_size
        pos2[1] -= step_size
        pos3[1] -= step_size

    return pos1, pos2, pos3

```

```
# 4. Initialize

# Initialize The Sinking Vortex Field
x = np.arange(x_start, x_end, quiver_step_size)
y = np.arange(y_start, y_end, quiver_step_size)
X, Y = np.meshgrid(x, y)
u, v = spring_stream(X, Y, x_source, y_source)

# Initialize the Robot's positions
pos1=robot_1_start
pos2=[(robot_1_start[0] + robot_2_offset[0]),(robot_1_start[1] + robot_2_offset[1])]
pos3=[(robot_1_start[0] + robot_3_offset[0]),(robot_1_start[1] + robot_3_offset[1])]

# Initialize output Plot
plt.figure(figsize=(6, 6))
plot_robot_position(pos1,pos2,pos3)


# Doing a rotation step prior to the main loop.

[u1, v1] = robot_readings(pos1[0],pos1[1], x_source,y_source)
[u2, v2] = robot_readings(pos2[0],pos2[1], x_source,y_source)
[u3, v3] = robot_readings(pos3[0],pos3[1], x_source,y_source)

[mag1, angle1] = composite_vector(u1,v1)
[mag2, angle2] = composite_vector(u2,v2)
[mag3, angle3] = composite_vector(u3,v3)

angle = (angle1+angle2+angle3)/3 +np.pi/2

pos1=robot_1_start
pos2=[pos1[0] + robot_offset*np.cos(angle),pos1[1] + robot_offset*np.sin(angle)]
pos3=[pos2[0] + robot_offset*np.cos(angle),pos2[1] + robot_offset*np.sin(angle)]


plot_robot_position(pos1,pos2,pos3) # Plot Robot Position




# 5. Main

for _ in range(1,100):
```

```
    # Check Break Criteria

    # Find u,v, angle and magnitude at each Robot

    [u1, v1] = robot_readings(pos1[0],pos1[1], x_source,y_source)
    [u2, v2] = robot_readings(pos2[0],pos2[1], x_source,y_source)
    [u3, v3] = robot_readings(pos3[0],pos3[1], x_source,y_source)

    [mag1, angle1] = composite_vector(u1,v1)
    [mag2, angle2] = composite_vector(u2,v2)
    [mag3, angle3] = composite_vector(u3,v3)

    # Plot Robot Position
    plot_robot_position(pos1,pos2,pos3)

    # Update Robot Position
    #angle = -(angle1 + angle2 + angle3)/3
    #slope_x = (mag2 - mag1) / robot_2_offset[0]
    slope_y1 = (mag2 - mag1) / robot_2_offset[1]
    slope_y2 = (mag3 - mag2) / robot_3_offset[1] - robot_2_offset[1]

    #angle = np.arctan2(slope_y,slope_x) #+ np.pi/2

    # Update Robot Position

    [pos1, pos2, pos3] = update_robot_position(pos1,pos2,pos3,slope_y1, slope_y2, robot_step_size)


# 6. Plot Outcome


plt.quiver(x, y, u, v)
plt.title(' The Sinking Vortex')
#plot_extra_features(x, y, u, v, x_source, y_source)
plt.show()

```


The Sinking Vortex
10
8
6
4
2
0
0
2
4
6
8
10

Lowest Magnitude Stagnation Point Finder

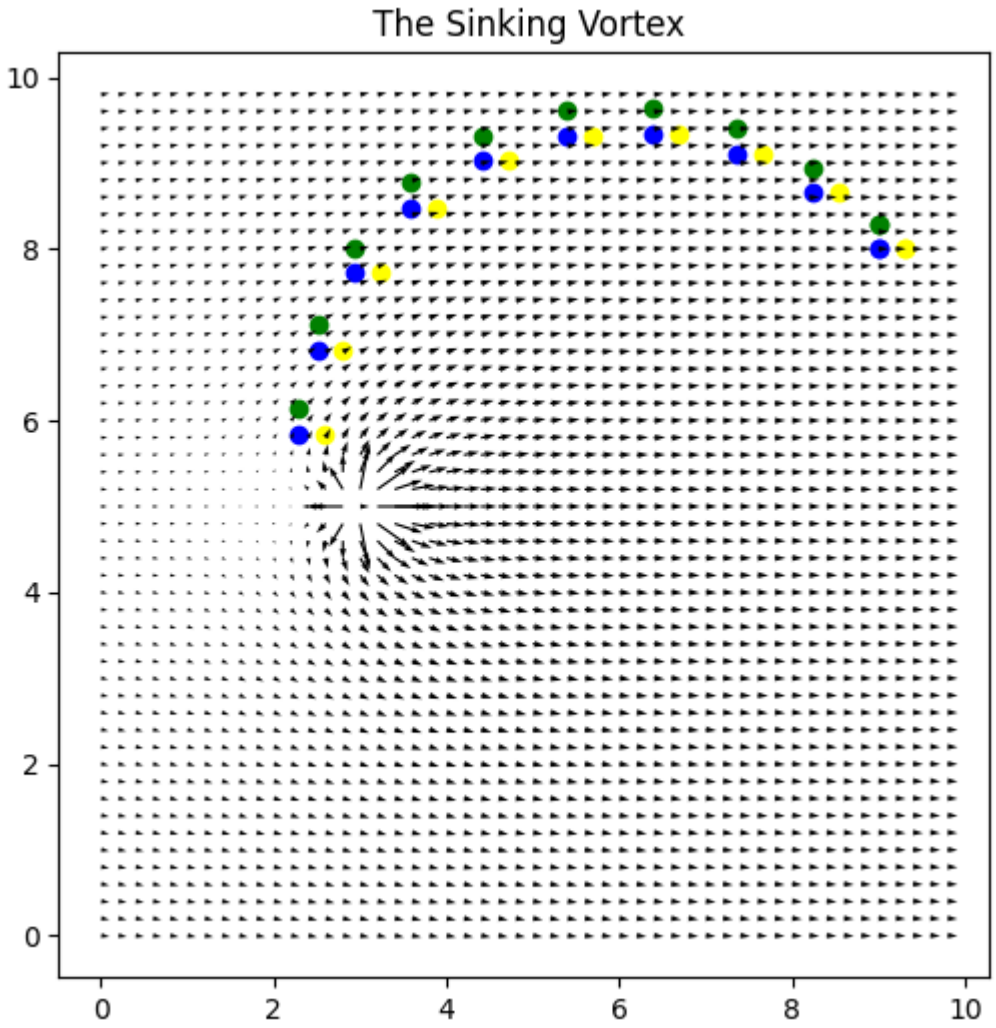
In [3]:

```python
## 1. Import Libraries
import numpy as np
import matplotlib.pyplot as plt

# 2. Define constants
x_start = y_start = 0
x_end = y_end = 10
quiver_step_size = 0.2

x_source = 3
y_source = 5

robot_1_start = [9,8]
robot_2_offset = [0.3,0]
robot_3_offset = [0,0.3]

robot_step_size = 0.5
step_size = 1



def spring_stream(x, y, x_source, y_source):

    # Establishing a Uniform Flow
    u = np.ones_like(x)
    v = np.zeros_like(x)


    # Calculating A Source
    x_centre = x - x_source
    y_centre = y - y_source
    r = np.sqrt(x_centre**2 + y_centre**2)
    r[r==0] = 1e-6

    # Adding the uniform flow to the spring source
    u = u + x_centre / r**2
    v = v + y_centre / r**2

    return u, v

def robot_readings(x, y, x_source, y_source):

    # Centring over sink
```

```
    x_centre = x - x_source
    y_centre = y - y_source

    # Calculate the distance r from the center
    dist = 0.8*np.sqrt((x-x_source)**2 + (y-y_source)**2)

    # Establishing a Uniform Flow
    u = np.ones_like(x)
    v = np.zeros_like(x)

    # Adding the uniform flow to the spring source
    u = u + x_centre / dist**2
    v = v + y_centre / dist**2

    return u, v

def composite_vector(u,v):

    mag = np.sqrt(u**2 + v**2)
    angle = np.arctan2(v,u)

    return mag, angle

def plot_robot_position(pos1,pos2,pos3):

    plt.scatter(pos1[0],pos1[1], marker='o', color='blue')
    plt.scatter(pos2[0],pos2[1], marker='o', color = 'yellow')
    plt.scatter(pos3[0],pos3[1], marker='o', color = 'green')


def update_robot_position(pos1,pos2,pos3, slope_y, slope_x, robot_step_size):

    pos1[0] += step_size*np.cos(angle)
    pos1[1] += step_size*np.sin(angle)
    pos2[0] += step_size*np.cos(angle)
    pos2[1] += step_size*np.sin(angle)
    pos3[0] += step_size*np.cos(angle)
    pos3[1] += step_size*np.sin(angle)

    return pos1, pos2, pos3

# 4. Initialize

```

```
# Initialize The Sinking Vortex Field
x = np.arange(x_start, x_end, quiver_step_size)
y = np.arange(y_start, y_end, quiver_step_size)
X, Y = np.meshgrid(x, y)
u, v = spring_stream(X, Y, x_source, y_source)

# Initialize the Robot's positions
pos1=robot_1_start
pos2=[(robot_1_start[0] + robot_2_offset[0]),(robot_1_start[1] + robot_2_offset[1])]
pos3=[(robot_1_start[0] + robot_3_offset[0]),(robot_1_start[1] + robot_3_offset[1])]

# Initialize output Plot
plt.figure(figsize=(6, 6))
plot_robot_position(pos1,pos2,pos3)

# 5. Main

for _ in range(1,11):
    # Check Break Criteria

    # Find u,v, angle and magnitude at each Robot

    [u1, v1] = robot_readings(pos1[0],pos1[1], x_source,y_source)
    [u2, v2] = robot_readings(pos2[0],pos2[1], x_source,y_source)
    [u3, v3] = robot_readings(pos3[0],pos3[1], x_source,y_source)

    [mag1, angle1] = composite_vector(u1,v1)
    [mag2, angle2] = composite_vector(u2,v2)
    [mag3, angle3] = composite_vector(u3,v3)

    # Plot Robot Position
    plot_robot_position(pos1,pos2,pos3)



    # Update Robot Position
    #angle = -(angle1 + angle2 + angle3)/3
    slope_x = (mag2 - mag1) / robot_2_offset[0]
    slope_y = (mag3 - mag1) / robot_3_offset[1]
    angle = np.arctan2(slope_y,slope_x) - np.pi/2

    [pos1, pos2, pos3] = update_robot_position(pos1,pos2,pos3,slope_y, slope_x, robot_step_size)

```

```
    if mag1**2 +mag2**2 + mag3**2 < 3.5 : # and slope_y**2<0.01:
        break

# 8. Getting to the stagnation point.

# 6. Plot Outcome


plt.quiver(x, y, u, v)
plt.title(' The Sinking Vortex')
#plot_extra_features(x, y, u, v, x_source, y_source)
plt.show()

```


The Sinking Vortex
10
8
6
4
2
0
0
2
4
6
8
10